

Esercizio 2

Sentiment Analysis con Transformer

Guida di studio: teoria, codice, risultati e domande d'esame

In una riga: un modello **DistilBERT** pre-addestrato, con una testa di classificazione aggiunta e addestrato in **fine-tuning** su **SST-2**, classifica frasi come positive o negative con **accuracy 89,8%** e **F1 0,902** su un test set di 872 frasi mai viste.

Indice

1. Il quadro in una pagina	3
2. La consegna e il dataset	3
3. Dal testo ai numeri: la tokenizzazione	5
4. Perché i transformer	5
5. Self-attention nel dettaglio	6
6. Il blocco transformer, BERT e DistilBERT	7
7. Il modello del progetto	9
8. Il fine-tuning	10
9. Come è organizzato il codice	11
10. Come è andato il training	11
11. Risultati sul test set	12
12. Analisi degli errori	13
13. Limiti e possibili miglioramenti	14
14. Domande d'esame probabili	15
15. Glossario	16

1. Il quadro in una pagina

Questa guida spiega l'esercizio 2 dall'inizio alla fine: cosa chiede la consegna, la teoria necessaria (attention, transformer, BERT), come è fatto il codice, come è stato addestrato il modello e come leggere i risultati. Tutti i numeri riportati sono misurati sul modello effettivamente addestrato.

Aspetto	Scelta nel progetto
Compito	Classificazione binaria di frasi: <i>positive / negative</i> (sentiment analysis)
Dataset	SST-2 (Stanford Sentiment Treebank): frasi da recensioni di film
Modello	DistilBERT <i>base uncased</i> : transformer encoder-only, 6 blocchi, 67,0 M parametri
Tecnica	Transfer learning con fine-tuning : tutti i pesi vengono aggiornati
Training	20.000 frasi, 2 epoche, AdamW, learning rate $2 \cdot 10^{-5}$, ~1,5 minuti su GPU
Selezione	Checkpoint con F1 migliore su un validation di 2.000 frasi ricavato dal train
Risultato	Test (872 frasi): accuracy 89,8% , F1 0,902 , precision 0,885, recall 0,919

Le 6 idee da portare all'esame

- La **self-attention** costruisce la rappresentazione di ogni parola come media pesata di tutte le altre: cattura dipendenze lontane e si calcola in parallelo, a differenza delle RNN.
- **BERT** è un transformer **encoder-only** bidirezionale, pre-addestrato a ricostruire parole mascherate: è fatto per *capire* un testo, non per generarlo.
- **DistilBERT** è un BERT compresso con *knowledge distillation*: metà dei blocchi, prestazioni molto vicine.
- Per classificare si usa il vettore del token speciale **[CLS]** seguito da una piccola testa lineare.
- **Fine-tuning**: learning rate piccolo e warmup, per adattare il modello senza distruggere ciò che sa già.
- Tre split con ruoli diversi: il **test** si guarda una sola volta, altrimenti il risultato è ottimistico.

2. La consegna e il dataset

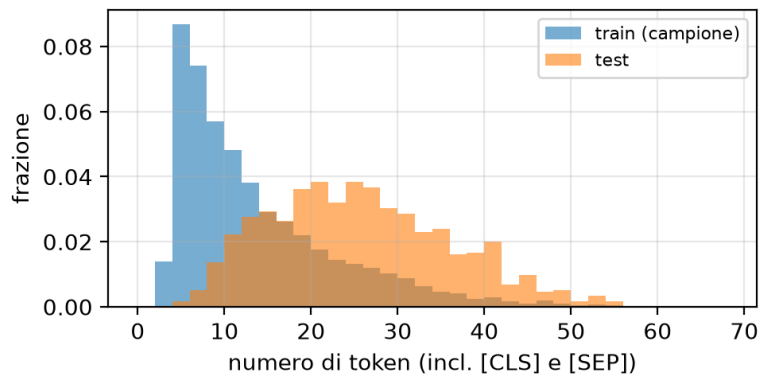
2.1 Cosa chiede l'esercizio

La traccia chiede di realizzare un sistema di *text classification* per frasi che possono essere considerate positive o negative, basato su **architetture transformer**, suggerendo i modelli di HuggingFace e dataset di sentiment analysis (per esempio da Kaggle).

Il progetto usa **SST-2** perché corrisponde esattamente al compito (frasi brevi con etichetta binaria) ed è il benchmark standard della sentiment analysis (fa parte di GLUE). Il codice accetta comunque anche un CSV Kaggle con `--dataset csv`.

2.2 SST-2

Lo **Stanford Sentiment Treebank** contiene frasi estratte da recensioni cinematografiche. Una sua particolarità è importante per capire i risultati: il **train** non contiene solo frasi complete, ma anche molte **sotto-frasi** (pezzi dell'albero sintattico delle stesse frasi), mentre validation e test ufficiali contengono frasi complete. Lo si vede dalla lunghezza in token:



Lunghezza delle frasi in token. Train: media 13,2, mediana 10. Test: media 25,2, massimo 55. Nessuna frase supera i 128 token usati come limite.

Split ufficiale SST-2	Frase	Etichette
train	67.349	disponibili
validation	872	disponibili
test	1.821	non pubbliche (tutte -1)

2.3 Tre split, tre ruoli

Il test ufficiale non ha etichette, quindi non si può usare per misurare nulla. Il progetto organizza i dati così:

Split nel progetto	Da dove viene	Frase	A cosa serve
train	train ufficiale meno 2.000 frasi	65.349 (se ne usano 20.000)	aggiornare i pesi
validation	2.000 frasi estratte a caso dal train	2.000	scegliere il checkpoint migliore
test	validation ufficiale	872 (428 neg., 444 pos.)	valutazione finale, una sola volta

Perché servono validation e test separati

Durante il training il Trainer valuta il modello ogni 250 step e tiene il checkpoint con l'F1 più alto. Scegliere il migliore tra tanti candidati su un insieme di dati significa adattarsi anche al rumore di quel particolare insieme: il suo punteggio diventa ottimistico. Il test, mai usato per decidere nulla, dà invece una stima onesta di come il modello si comporterà su frasi nuove.

È stato verificato che **nessuna delle 872 frasi del test compare nel train o nel validation** (confronto esatto delle frasi normalizzate). Il validation invece è ricavato dal train, che contiene sotto-frasi delle stesse recensioni: 25 frasi su 2.000 hanno un duplicato esatto nel train e molte altre sono parti di frasi presenti. È il motivo per cui l'accuracy sul validation (93,3%) è più alta di quella sul test (89,8%): non è un problema, perché il validation serve solo a confrontare checkpoint tra loro.

Le classi sono abbastanza bilanciate: nel train il 56% delle frasi è positivo, nel test il 51%. Per un compito bilanciato l'accuracy è una metrica sensata, ma si riportano comunque precision, recall e F1.

3. Dal testo ai numeri: la tokenizzazione

Una rete lavora su numeri, non su stringhe. Il **tokenizer** spezza la frase in unità del vocabolario e le trasforma in indici interi. DistilBERT usa **WordPiece** con un vocabolario di 30.522 unità e lavora in minuscolo (*uncased*).

Frase	Token prodotti
It is not bad at all, quite enjoyable	[CLS] it is not bad at all , quite enjoyable [SEP]
the unwatchable film	[CLS] the un ##watch ##able film [SEP]
a masterpiece of storytelling	[CLS] a masterpiece of storytelling [SEP]

- **Sotto-parole:** una parola rara viene spezzata in pezzi noti. *unwatchable* diventa un `##watch ##able`, dove `##` indica la continuazione di una parola. Così nessuna parola è davvero sconosciuta e il modello riconosce il prefisso negativo *un-*.
- **[CLS]** è un token speciale messo all'inizio di ogni frase: il suo vettore finale viene usato come riassunto dell'intera frase per la classificazione. **[SEP]** ne marca la fine.
- **Padding dinamico:** le frasi di un batch devono avere la stessa lunghezza, quindi le più corte vengono allungate con un token di padding (`DataCollatorWithPadding`, alla lunghezza della frase più lunga del batch).
- **Attention mask:** un vettore di 0/1 che dice al modello quali posizioni sono padding, così la self-attention le ignora.
- **Troncamento** a 128 token: nessuna frase di SST-2 si avvicina a questo limite.

4. Perché i transformer

4.1 Il limite delle RNN

Le reti ricorrenti leggono la frase una parola alla volta, aggiornando uno stato nascosto. Hanno due difetti: l'informazione di parole lontane deve sopravvivere a molti passaggi (e tende a svanire), e il calcolo è **sequenziale**, quindi non si parallelizza sulla lunghezza della frase. L'esempio delle slide è *"I grew up in France ... thus I speak ??"*: la parola da predire dipende da un termine molto distante.

4.2 L'idea dell'attention

Invece di comprimere tutto in uno stato, per costruire la rappresentazione di una parola la si confronta direttamente con **tutte** le altre e si decide quanto ognuna è rilevante. La distanza tra due parole non conta più: sono sempre a un solo passo di calcolo. E poiché i confronti sono indipendenti, si calcolano tutti insieme con moltiplicazioni di matrici, in parallelo.

5. Self-attention nel dettaglio

5.1 Query, key, value

Ogni token è un vettore (in DistilBERT di dimensione 768). Mettendo i vettori della frase nelle righe di una matrice X , la self-attention ne ricava tre versioni con tre matrici di pesi **apprese**:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

- **Query (Q)**: cosa sta cercando il token che vuole aggiornare la propria rappresentazione.
- **Key (K)**: come ogni token si presenta per essere trovato.
- **Value (V)**: l'informazione che ogni token consegna se viene scelto.

Il confronto tra query e key è un prodotto scalare; la softmax lo trasforma in pesi positivi che sommano a 1; l'output è la media dei value pesata con quei pesi:

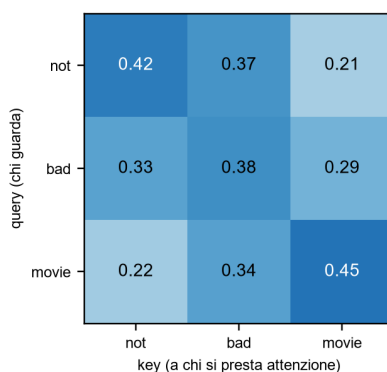
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- **Divisione per $\sqrt{d_k}$** : con vettori lunghi i prodotti scalari crescono in valore assoluto e la softmax si satura (quasi tutto il peso su un solo token, gradienti quasi nulli). Dividere per la radice della dimensione riporta i valori su una scala in cui la softmax resta morbida: è la *scaled* dot-product attention.
- **Costo**: si confronta ogni coppia di token, quindi il costo cresce con il quadrato della lunghezza. Per frasi brevi come quelle di SST-2 non è un problema.

5.2 Un esempio numerico

Per vedere il meccanismo senza dettagli superflui, prendiamo tre token con vettori di dimensione 2 e, **solo per semplicità**, $Q = K = V = X$ (nel modello vero ci sono le tre proiezioni apprese).

Token	Vettore x	Pesi di attention (riga = query)	Output = media pesata
not	(1,0; 0,0)	not 0,42 · bad 0,37 · movie 0,21	(0,72; 0,43)
bad	(0,8; 0,6)	not 0,33 · bad 0,38 · movie 0,29	(0,64; 0,52)
movie	(0,0; 1,0)	not 0,22 · bad 0,34 · movie 0,45	(0,49; 0,65)



Matrice dei pesi di attention dell'esempio: ogni riga è una softmax e somma a 1.

Per il token *bad*: i prodotti scalari con *not*, *bad*, *movie* divisi per $\sqrt{2}$ valgono 0,57, 0,71, 0,42; la softmax li trasforma nei pesi 0,33, 0,38, 0,29. Il nuovo vettore di *bad* è quindi una miscela che contiene una parte consistente di *not*: dopo l'attention la rappresentazione di *bad* "sa" di essere preceduta da una negazione. È il principio che permette al modello di gestire frasi come *not bad*.

5.3 Multi-head attention

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_{12}) W^O, \quad d_k = 768/12 = 64$$

Invece di una sola attention se ne calcolano 12 in parallelo (**teste**), ognuna con le proprie proiezioni di dimensione 64. Ogni testa può specializzarsi in un tipo di relazione (per esempio sintattica, o tra una negazione e l'aggettivo che modifica). Gli output vengono concatenati ($12 \times 64 = 768$) e rimescolati da una matrice W^O .

5.4 Positional encoding

La self-attention è **permutation equivariant**: se si mescolano le parole in input, gli output vengono mescolati allo stesso modo, ma non cambiano. Da sola tratta la frase come un *insieme* di parole, eppure *not bad* e *bad, not* non vogliono dire la stessa cosa. Per reintrodurre l'ordine si somma a ogni token un vettore che codifica la sua posizione. In BERT e DistilBERT questi vettori sono **appresi** (una tabella di 512 posizioni $\times$ 768), mentre nel Transformer originale erano funzioni seno/coseno fisse.

6. Il blocco transformer, BERT e DistilBERT

6.1 Il blocco encoder

Un blocco dell'encoder mette in sequenza due sotto-strati, ciascuno avvolto da una connessione residua e da una layer normalization:

$$X' = \text{LayerNorm}(X + \text{MHA}(X)), \quad Y = \text{LayerNorm}(X' + \text{FFN}(X'))$$

- **MHA**: la multi-head self-attention vista sopra (12 teste).
- **FFN**: una piccola rete feed-forward applicata a ogni token separatamente: $768 \rightarrow 3072 \rightarrow 768$ con attivazione **GELU**. L'attention mescola informazione *tra* i token, la FFN la elabora *dentro* ciascun token.
- **Connessione residua** ($X + \dots$): come in ResNet, il gradiente ha una strada diretta attraverso i blocchi, il che rende addestrabili reti profonde.
- **Layer normalization**: normalizza le feature di ogni singolo token (media 0, varianza 1, più scala e traslazione apprese). A differenza della batch norm non dipende dagli altri esempi del batch, quindi funziona bene con sequenze di lunghezza variabile e batch piccoli.
- **Dropout** 0,1 sulle attivazioni e 0,1 sui pesi di attention, come regolarizzazione.

6.2 Encoder, decoder e BERT

Famiglia	Come guarda il testo	Adatta a	Esempi
Encoder-only	Bidirezionale: ogni token vede tutti gli altri	Capire: classificazione, NER	BERT, DistilBERT
Decoder-only	Causale: ogni token vede solo i precedenti	Generare testo	GPT
Encoder-decoder	Encoder bidirezionale + decoder causale	Trasformare: traduzione, riassunto	T5, Transformer originale

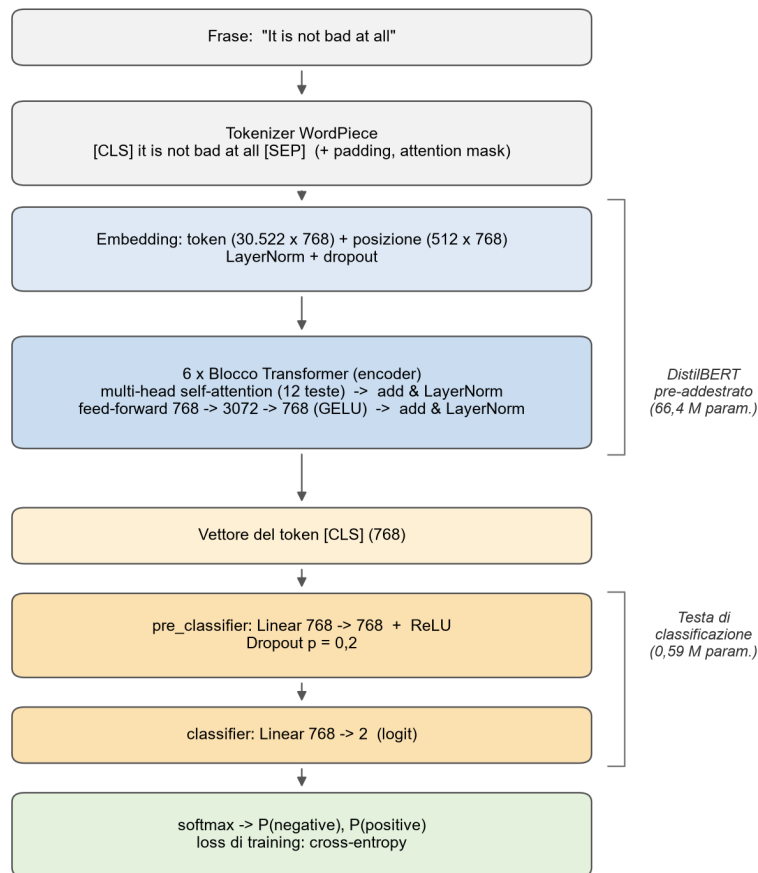
BERT è un encoder-only pre-addestrato su grandi quantità di testo con il **masked language modeling**: una parte dei token (il 15%) viene nascosta e il modello deve ricostruirla usando il contesto a sinistra e a destra. Per riuscirci deve imparare sintassi e significato: è questa conoscenza generale che il fine-tuning riusa. Il BERT originale aveva anche un secondo obiettivo (prevedere se due frasi sono consecutive).

6.3 DistilBERT e la knowledge distillation

DistilBERT è ottenuto con la **knowledge distillation**: un modello piccolo (lo *studente*) viene addestrato a imitare le distribuzioni di probabilità prodotte da BERT-base (l'*insegnante*), non solo le etichette corrette. Le probabilità “morbide” dell'insegnante contengono più informazione di una singola etichetta (per esempio quali parole sono plausibili alternative). Il risultato secondo gli autori: circa il 40% di parametri in meno e il 60% più veloce, conservando circa il 97% delle prestazioni di BERT.

	BERT-base	DistilBERT (usato qui)
Blocchi transformer	12	6
Dimensione nascosta / teste	768 / 12	768 / 12
Parametri	circa 110 M	66.955.010 (con la testa)

7. Il modello del progetto



Il percorso di una frase attraverso il modello.

`AutoModelForSequenceClassification` carica il corpo pre-addestrato di DistilBERT e aggiunge una **testa di classificazione** inizializzata a caso:

- prende il vettore finale del token **[CLS]** (768 numeri);
- lo passa in `pre_classifier` (lineare 768 → 768) seguito da ReLU e da un dropout con $p = 0,2$;
- `classifier` (lineare 768 → 2) produce due numeri, i **logit** delle classi.

La testa ha 592.130 parametri contro i 66.362.880 del corpo: meno dell'1%. Durante il training la softmax trasforma i logit in probabilità e la loss è la **cross-entropy**, cioè il logaritmo negativo della probabilità assegnata alla classe corretta:

$$p_c = \frac{e^{z_c}}{e^{z_{neg}} + e^{z_{pos}}}, \quad \mathcal{L} = -\log p_y$$

La loss vale 0 se il modello assegna probabilità 1 alla classe giusta e cresce senza limite quando è sicuro della classe sbagliata. All'inizio, con una testa casuale, le probabilità sono circa 50/50 e la loss vale circa $\ln 2 \approx 0,69$: è esattamente il valore misurato ai primi step (0,70).

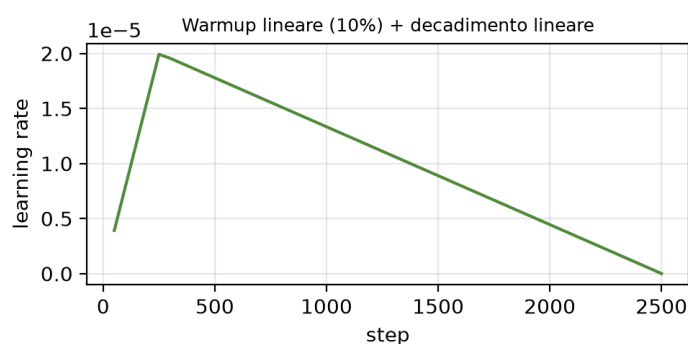
8. Il fine-tuning

8.1 Fine-tuning e feature extraction

Nel **transfer learning** si riusa un modello addestrato su un compito generale. Le slide del corso distinguono due modalità: la **feature extraction**, in cui il modello pre-addestrato resta congelato e si addestra solo la testa, e il **fine-tuning**, in cui si aggiornano tutti i parametri. Il progetto usa il fine-tuning: DistilBERT non ha mai visto il concetto di sentiment come etichetta, e adattare anche i blocchi interni dà risultati nettamente migliori di una testa su feature fisse.

8.2 Iperparametri effettivi

Iperparametro	Valore	Perché
Frase di training	20.000 (sottocampione)	training in pochi minuti; bastano per un buon risultato
Epoche / batch	2 / 16 → 2.500 step	20.000 / 16 = 1.250 step per epoca
Ottimizzatore	AdamW (β_1 0,9, β_2 0,999)	Adam con weight decay disaccoppiato
Learning rate	$2 \cdot 10^{-5}$	piccolo: si adatta il modello senza cancellare ciò che sa
Warmup	primo 10% degli step (250)	all'inizio la testa è casuale e i gradienti sono instabili
Schedule	lineare verso 0	passi sempre più piccoli verso la fine
Weight decay	0,01	regolarizzazione (penalità L2 sui pesi)
Gradient clipping	norma massima 1,0	evita aggiornamenti troppo grandi
Precisione	mixed precision (fp16)	calcoli più veloci e meno memoria sulla GPU
Valutazione / salvataggio	ogni 250 step	si tiene il checkpoint con F1 migliore sul validation



Learning rate misurato durante il training: sale linearmente nei primi 250 step, poi scende.

Perché un learning rate così basso

I pesi pre-addestrati sono già un ottimo punto di partenza. Con un learning rate alto (per esempio 10^{-3}) i primi aggiornamenti, guidati da una testa ancora casuale, stravolgerebbero le rappresentazioni imparate nel pre-training (*catastrophic forgetting*). Il warmup completa la protezione: nei primi step il learning rate è quasi zero, e cresce solo quando la testa ha iniziato a dare gradienti sensati.

9. Come è organizzato il codice

File	Cosa fa
config.py	Iperparametri in una dataclass Config; scelta del device (CUDA → MPS → CPU).
data.py	Carica SST-2 (o un CSV), crea i tre split, sottocampiona e tokenizza.
model.py	Crea DistilBERT con la testa a 2 classi e i nomi delle etichette.
train.py	Fine-tuning con il Trainer di HuggingFace; salva outputs/best_model.
evaluate.py	Classification report e matrice di confusione su validation o test.
predict.py	Classifica frasi passate da riga di comando, con la confidenza.

Il cuore della gestione degli split, in data.py:

```
raw = load_dataset("stanfordnlp/sst2")
split = raw["train"].train_test_split(test_size=cfg.val_size, seed=cfg.seed)
return DatasetDict(train=split["train"],          # addestramento
                   validation=split["test"],      # 2000 frasi: scelta del checkpoint
                   test=raw["validation"])        # 872 frasi: valutazione finale
```

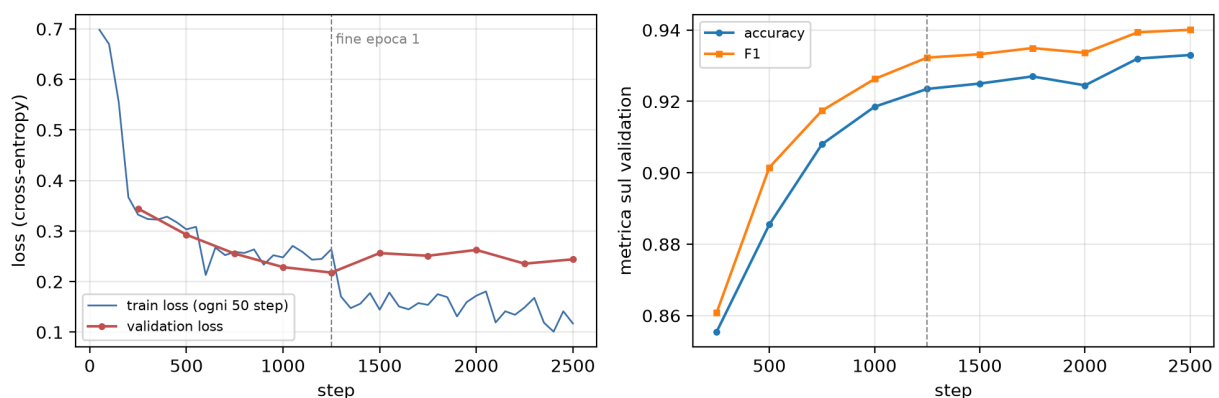
La configurazione del training, in train.py:

```
TrainingArguments(
    eval_strategy="steps", eval_steps=250, save_steps=250,
    load_best_model_at_end=True, metric_for_best_model="f1",
    learning_rate=2e-5, per_device_train_batch_size=16, num_train_epochs=2,
    weight_decay=0.01, warmup_steps=0.1,      # 0.1 = 10% degli step
    fp16=True)                                # solo su GPU CUDA
```

Comandi, da lanciare nella cartella es2_sentiment:

```
python src/train.py                # fine-tuning
python src/evaluate.py             # metriche sul validation
python src/evaluate.py --split test # valutazione finale
python src/predict.py "I really loved this movie"
```

10. Come è andato il training



Sinistra: loss di training (media ogni 50 step) e loss sul validation. Destra: accuracy e F1 sul validation. La linea tratteggiata separa le due epoche.

Step	Val. loss	Val. accuracy	Val. F1
250	0,344	85,5%	0,861
500	0,293	88,5%	0,902
750	0,255	90,8%	0,917
1.000	0,229	91,8%	0,926
1.250	0,218	92,3%	0,932
1.500	0,256	92,5%	0,933
1.750	0,251	92,7%	0,935
2.000	0,263	92,5%	0,934
2.250	0,235	93,2%	0,939
2.500	0,244	93,3%	0,940

10.1 Cosa si legge nei grafici

- **Apprendimento rapidissimo:** in 250 step la loss scende da 0,70 a circa 0,33 e l'accuracy sul validation è già all'85%. Il modello pre-addestrato “sa già” molto sul linguaggio: bisogna solo insegnargli a esprimerlo come sentiment.
- **Il salto all'inizio dell'epoca 2:** la loss di training crolla da circa 0,25 a circa 0,15 appena il modello rivede le stesse frasi. Sono frasi già incontrate: è un segno di memorizzazione, non di miglioramento generale.
- **Loss e accuracy divergono:** la loss sul validation tocca il minimo allo step 1.250 (0,218) e poi risale leggermente, mentre accuracy e F1 continuano a salire fino allo step 2.500. Il modello continua a classificare meglio, ma diventa **troppo sicuro** sugli esempi che sbaglia: la cross-entropy punisce molto un errore commesso con confidenza alta, l'accuracy no.
- **Checkpoint scelto:** step 2.500, F1 sul validation 0,9401. Con due sole epoche l'overfitting è lieve; con più epoche sarebbe il momento di fermarsi (*early stopping*).

11. Risultati sul test set

Il modello scelto è stato valutato **una sola volta** sulle 872 frasi del test.

True label	negative	375	53
	positive	36	408
		negative	positive
		Predicted label	

Matrice di confusione

Righe: classe vera. Colonne: classe predetta.
Considerando *positive* come classe positiva:

- **TP = 408:** positive riconosciute
- **TN = 375:** negative riconosciute
- **FP = 53:** negative scambiate per positive
- **FN = 36:** positive scambiate per negative

Totale errori: 89 su 872.

11.1 Le metriche, calcolate a mano

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN} \quad F_1 = \frac{2PR}{P + R}$$

Metrica (classe positive)	Calcolo	Valore
Accuracy	$(408 + 375) / 872$	89,79%
Precision	$408 / (408 + 53)$	0,8850
Recall	$408 / (408 + 36)$	0,9189
F1	$2 \cdot 0,885 \cdot 0,919 / (0,885 + 0,919)$	0,9017

- **Precision**: tra le frasi che il modello dice positive, quante lo sono davvero.
- **Recall**: tra le frasi davvero positive, quante il modello ne trova.
- **F1**: media armonica delle due; è alta solo se entrambe sono alte.

Classe	Precision	Recall	F1	Frase
negative	0,9124	0,8762	0,8939	428
positive	0,8850	0,9189	0,9017	444
media (macro)	0,8987	0,8975	0,8978	872

11.2 Come leggerli

- **Leggera preferenza per “positive”**: 53 negative sono state classificate positive contro 36 nel verso opposto, quindi recall alto sulle positive (0,919) e più basso sulle negative (0,876). È coerente con il training, dove il 56% delle frasi è positivo.
- **Validation 93,3% contro test 89,8%**: la differenza viene dal validation ricavato dal train (sotto-frasi delle stesse recensioni) e dall'averci scelto il checkpoint. Il numero da riportare è quello del test.
- **Risultato in linea con l'atteso**: DistilBERT su SST-2 arriva in letteratura intorno al 91% usando tutto il train (67.000 frasi) e più tempo di training; qui se ne usano 20.000 per 2 epoche.

12. Analisi degli errori

12.1 Frasi costruite a mano

Frase	Predizione	Commento
I absolutely loved this film	positive 99,9%	caso facile
the worst purchase I have ever made	negative 99,5%	caso facile, fuori dominio (acquisti)
It is not bad at all, quite enjoyable	positive 99,8%	contiene <i>bad</i> ma la negazione è gestita
predictable but the acting saved it	negative 75,3%	sentiment misto: confidenza bassa

La frase con *not bad* è l'esempio classico in cui un modello a “sacco di parole” sbaglia, perché vede la parola negativa *bad*. Il transformer la classifica correttamente con grande sicurezza: le rappresentazioni contestuali, costruite con la self-attention, combinano *bad* con la negazione che la precede (sezione 5.2). La frase ambigua riceve invece una confidenza molto più bassa: in questo

caso il modello “segnala” l'incertezza.

12.2 Errori reali sul test

Sul test il modello sbaglia 89 frasi. Un dato interessante è la confidenza: in media 97,9% sulle risposte corrette, ma ancora 90,4% su quelle sbagliate. Il modello è **troppo sicuro di sé** quando sbaglia (è *mal calibrato*), in accordo con la risalita della loss sul validation vista nella sezione 10. Gli errori più sicuri sono istruttivi:

Frase del test (etichetta: negative)	P(positive)
it's somewhat clumsy and too lethargically paced -- but its story about a mysterious creature with psychic abilities offers a solid build-up, a terrific climax, and some nice chills along the way.	99,8%
moretti's compelling anatomy of grief and the difficult process of adapting to loss.	99,7%
this riveting world war ii moral suspense story deals with the shadow side of american culture : racial prejudice in its ugly and diverse forms.	99,7%
although huppert's intensity and focus has a raw exhilaration about it, the piano teacher is anything but fun.	99,7%
manages to show life in all of its banality when the intention is quite the opposite.	99,7%

- **Sentiment misto:** la prima frase critica il ritmo ma elogia costruzione, finale e brividi. L'etichetta negativa è difendibile, ma il testo è in prevalenza positivo.
- **Parole positive, giudizio negativo:** *compelling, riveting* descrivono l'argomento, mentre il giudizio negativo è implicito o sta nel tono.
- **Etichette discutibili:** alcune frasi sono ambigue anche per un lettore umano. SST-2 ha un certo rumore nelle etichette, e una parte degli errori non è davvero un errore del modello.

13. Limiti e possibili miglioramenti

Limite	Possibile intervento
Usate solo 20.000 frasi su 65.000 e 2 epoche	Usare tutto il train con early stopping sul validation
Modello compresso	BERT-base o RoBERTa: più lenti ma in genere 1–3 punti migliori
Confidenza troppo alta sugli errori	Calibrazione (per esempio temperature scaling sul validation)
Validation simile al train (sotto-frasi)	Validation da frasi complete, tenendo insieme le sotto-frasi della stessa recensione
Dominio: recensioni di film	Su recensioni di prodotti o tweet le prestazioni possono calare: servono dati del nuovo dominio
Classificazione binaria	Frasi neutre forzate in una delle due classi: aggiungere una classe neutra (SST-5 ha 5 livelli)

14. Domande d'esame probabili

1. Perché un transformer e non una RNN?

La self-attention collega direttamente ogni coppia di parole, quindi gestisce dipendenze lontane, e si calcola in parallelo su tutta la frase. Una RNN elabora una parola alla volta e l'informazione lontana tende a svanire.

2. Cosa sono query, key e value?

Tre proiezioni lineari apprese dello stesso input. La query di un token si confronta (prodotto scalare) con le key di tutti i token; la softmax dei punteggi dà i pesi con cui si sommano i value.

3. Perché si divide per $\sqrt{d_k}$?

Con vettori di dimensione grande i prodotti scalari hanno valori ampi, la softmax si satura e i gradienti diventano minuscoli. La divisione mantiene i punteggi su una scala stabile.

4. Perché serve il positional encoding?

La self-attention non dipende dall'ordine dei token (è permutation equivariant). Sommando un vettore di posizione a ogni token il modello distingue *not bad* da *bad, not*. In BERT è appreso.

5. Perché BERT per classificare e non GPT?

BERT è encoder-only e bidirezionale: ogni token vede tutta la frase, che è ciò che serve per capirla. I decoder come GPT guardano solo i token precedenti perché sono fatti per generare.

6. Cos'è il token [CLS]?

Un token speciale aggiunto all'inizio della frase. Attraverso la self-attention il suo vettore finale raccoglie informazione da tutta la frase e viene usato come input della testa di classificazione.

7. Cos'è DistilBERT?

Una versione di BERT con 6 blocchi invece di 12, ottenuta con knowledge distillation: lo studente impara a imitare le probabilità di BERT. Circa 40% più piccolo e 60% più veloce, con circa il 97% delle prestazioni.

8. Fine-tuning o feature extraction?

Fine-tuning: si aggiornano tutti i pesi, non solo la testa. Il modello pre-addestrato non conosce il sentiment come etichetta, e adattare i blocchi interni dà risultati migliori.

9. Perché learning rate $2 \cdot 10^{-5}$ e warmup?

Per non distruggere le rappresentazioni pre-addestrate (catastrophic forgetting). Il warmup evita aggiornamenti grandi all'inizio, quando la testa è casuale e i gradienti sono instabili.

10. Perché validation e test separati?

Il validation si usa per scegliere il checkpoint, quindi il suo punteggio è ottimistico. Il test non influenza nessuna scelta e dà una stima onesta. Qui: 93,3% sul validation, 89,8% sul test.

11. Calcola la precision dalla matrice di confusione.

Precision = $TP / (TP + FP) = 408 / (408 + 53) = 0,885$. Recall = $408 / (408 + 36) = 0,919$.

12. Perché la loss di validation risale mentre l'accuracy migliora?

Il modello sbaglia meno esempi ma quelli che sbaglia li sbaglia con più sicurezza. La cross-entropy penalizza molto gli errori sicuri, l'accuracy conta solo giusto/sbagliato. È un primo segno di overfitting.

13. Come gestisce la negazione?

La rappresentazione di ogni parola è contestuale: la self-attention mescola in *bad* l'informazione di *not*. "It is not bad at all" è classificata positiva al 99,8%.

14. Che cos'è WordPiece?

Un tokenizer a sotto-parole: le parole rare vengono spezzate in pezzi del vocabolario (*unwatchable* → un *##watch ##able*), così non ci sono parole sconosciute e il vocabolario resta di circa 30.000 unità.

15. Layer norm o batch norm?

La layer norm normalizza le feature di ogni token indipendentemente dal resto del batch; la batch norm usa le statistiche del batch. Con sequenze di lunghezza variabile e padding la layer norm è più stabile.

16. Cosa miglioreresti?

Tutto il train con early stopping, un modello più grande (BERT-base, RoBERTa), la calibrazione delle probabilità e dati del dominio di utilizzo reale.

15. Glossario

Termine	Significato
Attention mask	Vettore 0/1 che indica le posizioni di padding da ignorare.
Checkpoint	Copia dei pesi salvata durante il training.
[CLS]	Token iniziale il cui vettore finale riassume la frase.
Cross-entropy	Loss di classificazione: $-\log$ della probabilità della classe corretta.
Encoder-only	Transformer in cui ogni token vede tutti gli altri (bidirezionale).
Fine-tuning	Addestramento di tutti i pesi di un modello pre-addestrato sul nuovo compito.
GELU	Funzione di attivazione morbida simile a ReLU, usata nei transformer.
Knowledge distillation	Addestrare un modello piccolo a imitare le uscite di uno grande.
Logit	Punteggio grezzo di una classe, prima della softmax.
Masked language modeling	Pre-training: ricostruire token nascosti usando il contesto.
Mixed precision (fp16)	Calcoli a 16 bit dove possibile, per velocità e memoria.
Multi-head attention	Più attention in parallelo con proiezioni diverse, poi concatenate.
Self-attention	Ogni token si aggiorna come media pesata dei value di tutti i token.
SST-2	Stanford Sentiment Treebank, versione binaria: frasi di recensioni di film.
Warmup	Fase iniziale in cui il learning rate cresce gradualmente da zero.
WordPiece	Tokenizer a sotto-parole usato da BERT.